WEEK 2: SQL LOGIC + MODELLING (POSTGRESQL)

PostgreSQL After SQLite

- SQLite is embedded and file-based.
- PostgreSQL is server-based and supports concurrent users.
- PostgreSQL has stronger support for constraints, joins, and advanced SQL.
- Docker gives the class a consistent setup.
- The VS Code extension lets you run queries inside the editor.

Our Example Tables: weather and cities

From the official PostgreSQL tutorial:

cities

name	location
San Francisco	(-194.0, 53.0)

weather

city	temp_lo	temp_hi	prcp	date
San Francisco	46	50	0.25	1994-11-27
San Francisco	43	57	0.0	1994-11-29
Hayward	37	54	NULL	1994-11-29

Subqueries

- A subquery is a query inside another query.
- In the PostgreSQL tutorial, it solves a query you cannot write directly.

```
SELECT city
FROM weather
WHERE temp_lo = (
  SELECT max(temp_lo) FROM weather
);
```

Result: San Francisco (temp_lo = 46)

CTEs

- A CTE is a named intermediate result set.

```
WITH regional_sales AS (
  SELECT region, SUM(amount) AS total_sales FROM orders GROUP BY region
),
top_regions AS (
  SELECT region FROM regional_sales
  WHERE total_sales > (SELECT SUM(total_sales) / 10 FROM regional_sales)
)
SELECT region FROM top_regions;
```

JOIN Types

- `INNER JOIN`: matching rows only.
- `LEFT JOIN`: all left rows, plus matches from the right.
- `RIGHT JOIN`: all right rows, plus matches from the left.
- `FULL OUTER JOIN`: rows from both sides.
- Prefer explicit `JOIN ... ON ...` syntax.

LEFT JOIN

```
SELECT w.city, w.temp_lo, w.temp_hi, c.location
FROM weather w
LEFT JOIN cities c ON w.city = c.name;
```

- Missing matches on the right side produce `NULL` values.

Why Relationships Matter: BI Tools

- Relationships are **critical** for Business Intelligence tools.
- Power BI, Tableau, and Looker rely on properly defined relationships.
- If you get this wrong, BI tools will complain loudly!

Power BI Example:

- Without proper relationships, you get the dreaded error:

  > "A relationship between tables is expected but not defined"

- Or worse: **wrong aggregations** and **duplicate counts**
- Power BI's model view requires clear many-to-one definitions.

Many-to-One Relationships

- Many rows in one table can point to one row in another table.
- PostgreSQL's foreign-key tutorial uses this pattern.

---

cities_fk

name	location
San Francisco	(-194.0, 53.0)
Hayward	(10.0, 20.0)

weather_fk (many weather records → one city)

city	temp_lo	temp_hi	date
San Francisco	46	50	1994-11-27
San Francisco	43	57	1994-11-29
Hayward	37	54	1994-11-29

The REFERENCES Keyword

- `REFERENCES` creates a **foreign key constraint**.
- It enforces **referential integrity**: the referenced row must exist.
- PostgreSQL will reject inserts/updates that violate this rule.

```
CREATE TABLE cities (
  name varchar(80) PRIMARY KEY  -- The referenced column
);
CREATE TABLE weather (
  city varchar(80) REFERENCES cities(name),  -- FK constraint
  date date
);
```

- `weather.city` can only contain values that exist in `cities.name`.
- Trying to insert `weather` with `city = 'Paris'` fails if Paris doesn't exist.

Many-to-Many: The Tables

Before we connect them, let's see what we're working with:

---

products

product_no	name	price
1	Laptop	1200.00

product_no	name	price
2	Mouse	20.00
3	Monitor	300.00

orders_docs

order_id	shipping_address
1001	London
1002	Manchester

How do we connect them? A product can be in many orders, an order can have many products.

Many-to-Many Relationships

- PostgreSQL docs use a **junction table** for this pattern.
- One order can contain many products.
- One product can appear in many orders.
- The junction table holds the relationship + extra attributes (quantity).

```
CREATE TABLE order_items (
  product_no integer REFERENCES products,
  order_id integer REFERENCES orders_docs,
  quantity integer,
  PRIMARY KEY (product_no, order_id)
);
```

---

order_items (the junction table)

product_no	order_id	quantity
1	1001	1
2	1001	2
3	1002	1

Querying A Junction Table

```
SELECT oi.order_id, p.name, oi.quantity
FROM order_items oi
JOIN products p ON p.product_no = oi.product_no
ORDER BY oi.order_id, p.product_no;
```

Normalization

- Normalization reduces redundancy.
- It also reduces update, insert, and delete anomalies.
- Start with a clean normalized design.
- Denormalize only when a measured performance problem justifies it.

What Are Functional Dependencies?

A **functional dependency** means: if you know X, you can determine Y.

Notation: `X → Y` (X determines Y)

Example: In a student table:

- `student_id → student_name` (knowing the ID gives you the name)
- `student_id → email` (knowing the ID gives you the email)

Non-example:

- `student_name → student_id` (two students can have the same name)

Understanding FDs is the **mathematical foundation** of normalization.

Functional Dependencies: Visual Example

student_id	student_name	course_id	course_name	grade
1	Alice	101	Math	90
1	Alice	102	Physics	85
2	Bob	101	Math	88

Dependencies we can identify:

- `student_id → student_name` (Alice is always student 1)
- `course_id → course_name` (Math is always course 101)
- `(student_id, course_id) → grade` (the full key determines the grade)

Primary Keys And Dependencies

- Start by asking: what uniquely identifies one row?
- In enrollments, the natural key is `(student_id, course_id)`.

```
student_id -> student_name
course_id -> course_name, lecturer_id
(student_id, course_id) -> grade
```

First Normal Form (1NF)

- 1NF means **atomic values** in each column.
- Avoid lists, repeated columns, or comma-separated values.

Bad (violates 1NF):

student_id	student_name	courses
1	Alice	Math, Physics
2	Bob	Math

Good (1NF compliant):

student_id	course_id
1	101
1	102
2	101

- 1NF is a design rule, not a feature switch.

Second Normal Form (2NF)

- Table must be in 1NF.
- Non-key columns must depend on the **whole** composite key.

Bad (violates 2NF): PK = (student_id, course_id)

student_id	course_id	student_name	course_name	grade
1	101	Alice	Math	90
1	102	Alice	Physics	85

- `student_name` depends only on `student_id` (partial dependency!)
- `course_name` depends only on `course_id` (partial dependency!)

2NF Solution: Split The Tables

students

student_id	student_name
1	Alice
2	Bob

courses

course_id	course_name
101	Math
102	Physics

enrollments (PK = student_id, course_id)

student_id	course_id	grade
1	101	90
1	102	85

Third Normal Form (3NF)

- 3NF removes **transitive dependencies**.
- A transitive dependency: A → B → C (A determines C through B)

Bad (violates 3NF):

course_id	course_name	lecturer_id	lecturer_name
101	Math	L1	Dr. Smith
102	Physics	L1	Dr. Smith
103	Chemistry	L2	Dr. Jones

- `course_id → lecturer_id → lecturer_name` (transitive!)
- If Dr. Smith changes name, we must update multiple rows.

3NF Solution: Remove Transitive Dependency

course_id	course_name	lecturer_id
101	Math	L1
102	Physics	L1
103	Chemistry	L2

lecturer_id	lecturer_name
L1	Dr. Smith
L2	Dr. Jones

Now each fact is stored in exactly one place.

Boyce-Codd Normal Form (BCNF)

- BCNF is stricter: **every determinant must be a candidate key**.
- 3NF allows non-key determinants if they determine only prime attributes.

Scenario: Each project has exactly one manager. An employee can work on many projects.

Bad (violates BCNF): PK = `(employee_id, project_id)`

employee_id	project_id	manager_id
E1	P1	M1
E2	P1	M1
E1	P2	M2

- `manager_id → project_id` (M1 always manages P1)
- `manager_id` is a determinant but not a candidate key → BCNF violation!
- Problem: if M1 is replaced, every row for P1 must be updated.

BCNF Solution

Split so that "M1 manages P1" lives in exactly one place:

manager_id	project_id
M1	P1
M2	P2

employee_id	manager_id
E1	M1
E2	M1
E1	M2

Now every determinant (`manager_id`, `employee_id + manager_id`) is a candidate key.

How To Normalize Mathematically

1. List the attributes.
2. Write the functional dependencies.
3. Find the candidate key(s).
4. Remove repeating groups to reach 1NF.
5. Remove partial dependencies to reach 2NF.
6. Remove transitive dependencies to reach 3NF and BCNF.

Indexes

- An index is a data structure that speeds up lookups.
- A composite index can help with filtering and sorting together.

```
CREATE INDEX idx_orders_perf_cust_amount
ON orders_perf(cust_id, amount);

SELECT * FROM orders_perf
WHERE cust_id = 42
ORDER BY amount
LIMIT 5;
```

When To Add Indexes

- Good candidates: join keys, filter columns, foreign keys.
- Indexes can also support matching `ORDER BY` clauses.
- They are not free: extra storage and slower writes.
- Add them when a real query pattern needs them.

Summary

- Subqueries let one query depend on another.
- CTEs improve readability for multi-step SQL.
- Foreign keys model many-to-one relationships.
- Junction tables model many-to-many relationships.
- Primary keys and functional dependencies drive normalization.
- 1NF, 2NF, 3NF, and BCNF reduce different kinds of anomalies.
- Indexes improve performance for common access patterns.

EXERCISES

Exercise 1

- Use `books` and `authors` from `week2.sql`.
- Find books priced above the average price.
- Find books with titles longer than the average title length.
- Find books written by authors from the top author country.

Exercise 2

- Rewrite the Exercise 1 queries with CTEs.

- Compare readability with the subquery versions.
- Which version would you rather maintain later?

Exercise 3

- Use `LEFT JOIN` to list every author and a book count.
- Identify authors with no books.
- Explain why `COUNT(b.id)` returns `0` for those authors.

Exercise 4

- Start from `student_courses_denorm` in `week2.sql`.
- Create `students_norm`, `courses_norm`, and `enrollments_norm`.
- Migrate the data into the normalized tables.
- Explain whether your result reaches 2NF only or also 3NF.

Exercise 5

- Use `orders_perf` from `week2.sql`.
- Create an index on `cust_id`.
- Create a composite index on `(cust_id, amount)`.
- Run `EXPLAIN` on the filter and sort queries.

Reflection

1. What is the practical difference between a subquery and a CTE?
2. What is the difference between many-to-one and many-to-many?
3. What do `NULL` values in a `LEFT JOIN` usually tell you?
4. What is the difference between 3NF and BCNF?

References: Where the Examples Come From

All examples in this slide deck are drawn from PostgreSQL's official documentation and our `week2.sql` script.

PostgreSQL Official Tutorials:

- `weather` and `cities` tables — PostgreSQL tutorial on joins and subqueries
- `REFERENCES` keyword pattern — PostgreSQL foreign key documentation

---

Our `week2.sql` Script:

- `books` and `authors` — Exercise dataset for subqueries and CTEs
- `orders` — CTE example with regional sales
- `products`, `orders_docs`, `order_items` — Many-to-many relationship pattern

- `student_courses_denorm` → split into `students_norm`, `courses_norm`, `enrollments_norm` — Normalization challenge
- `orders_perf` — Index performance examples

Conceptual Examples:

- Student/course/lecturer tables — Demonstrate 1NF, 2NF, 3NF, BCNF violations and solutions
- Functional dependencies — Annotated with real table patterns

All examples can be found and executed in the PostgreSQL environment or adapted to SQLite (except `FULL OUTER JOIN` for some version of SQLite).